TripMind

Architecture Reference

v3.1 — API + Scraper Dual-Path Architecture

March 12, 2026

React 19 + Vite • Express.js • FastAPI + Playwright • Claude AI

11 verification agents • 4 optional API integrations • 6 web scrapers • 5 free APIs

Contents

1 Project Overview **2**
- 1.1 What is TripMind? . . . 2
- 1.2 Three-Tier Architecture . . . 2
- 1.3 End-to-End User Flow . . . 2
- 1.4 Agent Invocation Conditions . . . 4

2 Component Reference **5**
- 2.1 Frontend (React 19 + Vite) . . . 5
- 2.2 Node.js Backend (Express) . . . 6
- 2.3 Python Verification Service (FastAPI) . . . 6
- 2.4 Verification Agents . . . 6
- 2.5 API Clients (Optional, Key Required) . . . 7
- 2.6 Scrapers (Default, No Key Required) . . . 7
- 2.7 Free APIs (No Key Required) . . . 7

3 System Overview **8**

4 Debug-Mode Execution Flow **9**
- 4.1 Verification Request Flow . . . 9
- 4.2 Function Call Trace (Hotel Agent Example) . . . 10

5 API vs Scraper Decision Reference **11**
- 5.1 Dual-Path Agents . . . 11
- 5.2 Free-Only Agents . . . 11
- 5.3 Environment Variable Quick Reference . . . 11
- 5.4 Data Contract Summary . . . 12

6 Frontend Data Quality Controls **13**
- 6.1 Search Links Filtering . . . 13
- 6.2 Price Trends Frontend Fallback . . . 13
- 6.3 Budget Selection System . . . 13
- 6.4 Booking Link Specificity . . . 13

A Detailed Agent Reference **15**
- A.1 Weather Agent . . . 15
- A.2 Safety Agent . . . 15
- A.3 Seasonal Agent . . . 15
- A.4 Currency Agent . . . 16
- A.5 Hotel Agent . . . 16
- A.6 Activity Agent . . . 16
- A.7 Review Agent . . . 17
- A.8 Flight Agent . . . 17
- A.9 Place Agent . . . 17
- A.10 Link Agent . . . 18
- A.11 Price Trends Agent . . . 18

B Orchestrator Internals **19**
- B.1 Dispatch Pipeline . . . 19
- B.2 Server-Sent Events (SSE) Streaming . . . 19
- B.3 Verification Score . . . 20
- B.4 Parameter Pass-Through . . . 20

1 Project Overview

1.1 What is TripMind?

TripMind is an AI-powered travel planning application that generates personalized, multi-day trip itineraries using Anthropic's Claude large language model and then independently verifies every detail — hotel prices, flight costs, activity availability, place existence, weather forecasts, safety advisories, currency rates, and more — through 11 concurrent verification agents. The key architectural insight is the **dual-path design**: each verification agent can use either a paid official API (Amadeus, Booking.com, Viator, Google Places) when the user provides a key, or fall back to free web scraping and public APIs, meaning the app is fully functional at zero cost beyond Claude API credits.

1.2 Three-Tier Architecture

TripMind consists of three independently running services:

1. **Frontend** (React 19 + Vite, port 5173) — A 9-step trip wizard that collects preferences (destinations, dates, budget, vibe, dietary needs, accessibility, children ages, passport country, etc.) and displays the generated itinerary across 7 tabs: Map, Itinerary, Stays, Flights, Budget, Suggestions, and Tips.
2. **Backend** (Node.js Express, port 3001) — An API gateway that proxies requests to the Claude API for trip generation and to the Python service for verification. It handles rate limiting, Server-Sent Events (SSE) streaming, and serves the production build.
3. **Verification Service** (Python FastAPI + Playwright, port 8000) — Runs 11 verification agents in parallel using `asyncio.gather()`. Each agent checks a SQLite cache first, then tries its data sources in priority order, and emits results back via SSE as they complete. Playwright provides headless Chromium for web scraping; httpx + BeautifulSoup handle lighter scraping; and five free public APIs require no keys at all.

1.3 End-to-End User Flow

This section traces a complete user interaction from opening the app to seeing verified results.

Phase 1: Trip Planning Form

1. User opens `http://localhost:5173` in the browser
2. The `FormSteps.jsx` component renders a 9-step wizard:
   - **Step 1** — Origin city (where you are traveling from)
   - **Step 2** — Destination(s) (one or more cities/countries)
   - **Step 3** — Travel dates (check-in, check-out)
   - **Step 4** — Travelers (adults, children with ages)
   - **Step 5** — Budget (total budget, currency, max nightly hotel rate)
   - **Step 6** — Trip vibe (adventure, relaxation, cultural, nightlife, etc.)
   - **Step 7** — Extras (dietary restrictions, accessibility needs, pet-friendly, passport country)
   - **Step 8** — Activity level (relaxed, moderate, packed)
   - **Step 9** — Review all selections before generating
3. All form state is managed by `TripContext.jsx` (React Context API)
4. User clicks "Generate Trip" on the review step

Phase 2: AI Trip Generation (Claude API)

1. `TripContext` calls `askClaudeStream()` from `src/api.js`
2. Frontend sends `POST /api/chat/stream` to Express (:3001)
3. Express proxies the request to the Anthropic Claude API with a structured prompt built by `src/prompts.js`
4. **Two-call strategy** to avoid token limits:
   - **Call 1 (Skeleton)**: Claude generates the trip structure — destinations, daily overview, budget breakdown, hotel/restaurant/activity names, booking URLs, travel tips
   - **Call 2 (Day-by-Day)**: Claude fills in the detailed day-by-day itinerary for each destination, using the skeleton as context
5. Responses stream back via SSE; the `LoadingView.jsx` component shows animated progress with streaming text fragments
6. Claude's response is parsed as JSON and stored in `TripContext` state
7. The UI transitions from the form to the `TripGuide.jsx` multi-tab results view

Phase 3: Verification (11 Concurrent Agents)

1. Immediately after Claude's response is received, `TripContext` calls `requestVerification()`
2. Frontend sends `POST /api/verify/stream` with the complete trip plan + form data to Express
3. Express proxies to Python FastAPI (`POST /verify/stream` on port 8000)
4. The FastAPI endpoint calls `run_verification_streaming(req, on_result)` in `orchestrator.py`
5. The orchestrator calls `_build_agent_coros(req)` which constructs up to 20+ coroutines (some agents run once per destination):
   - Weather, Safety, Seasonal, Hotels: **one per destination**
   - Flights, Price Trends: **primary destination only**
   - Places, Activities, Reviews, Links: **batch** (all items from the plan)
   - Currency: **single call** with all destinations
6. All coroutines launch simultaneously via `asyncio.gather()`
7. Each agent independently: checks cache → tries API (if key set) → tries scraper → falls back to search URLs
8. As each agent completes, the orchestrator calls `on_result()` which sends an SSE event back through Express to the browser
9. `VerificationBar.jsx` updates the progress bar in real time (e.g., "7/11 agents complete")
10. Results populate the tabs: weather appears in Tips, hotel prices appear in Stays and Budget, flight prices in Flights, etc.

Phase 4: User Interaction with Results

1. User browses the 7-tab results view:
   - **Map** — Interactive map with pinned locations (verified via Places agent)
   - **Itinerary** — Day-by-day plan with activities, times, restaurant suggestions
   - **Stays** — Verified hotel options with prices, ratings, and booking links; "Search Links" fallback entries are filtered out
   - **Flights** — Verified flight options with airline, price, duration; AI estimates shown when scraper fails
   - **Budget** — Interactive budget breakdown; user can select different hotel/flight

options and see live price delta badges (green = savings, red = more expensive) with automatic total recalculation
- **Suggestions** — Alternative restaurants and activities
- **Tips** — Weather forecast, safety advisory, seasonal insights, currency rates, visa requirements

2. User can **swap hotels/restaurants** via the Swap Modal (triggers on-demand searches through `/api/hotels/search` or `/api/restaurants/alternatives`)
3. User can **toggle currency** via CurrencyToggle (client-side conversion using verified rates)
4. User can **export to PDF** via the PdfExport button (lazy-loads `@react-pdf/renderer`, ∼1.5 MB)

1.4 Agent Invocation Conditions

Not all agents run for every trip. The orchestrator decides which agents to invoke based on the trip plan content and form data:

Agent	Condition	How Many Instances
Weather	Always (destinations exist)	One per destination city
Safety	Always (destinations exist)	One per destination city
Seasonal	Always (destinations exist)	One per destination city
Currency	Always (destinations exist)	Single call (all destinations)
Hotels	Plan contains accommodation locations	One per accommodation city
Flights	Plan contains origin + destination	Single call (primary destination)
Activities	Plan contains named activities	Single batch call (up to 15 activities)
Reviews	Plan contains named places/restaurants	Single batch call (up to 10 places)
Places	Plan contains named attractions	Single batch call (up to 20 places)
Links	Plan contains URLs	Single batch call (up to 50 URLs)
Price Trends	Plan contains origin + destination + dates	Single call (primary destination)

Key rules:

- If the Python verification service is not running, the app works fully with AI estimates only (graceful degradation)
- If an agent times out (45 s standard, 60 s for price trends), it returns an error result but does not block other agents
- If all data sources fail for an agent, it returns "Search Links" fallback entries which the frontend filters out
- Agents with cached results return immediately without making any network calls
- The `children_ages` array is passed through to Hotels (group size) and Flights (passenger count); the `passport_country` field is passed to Safety (advisory context)

2 Component Reference

Every file in TripMind is listed below, grouped by layer. **API Path** = official API used when the corresponding `.env` key is set; **Scraper Path** = default free path (no key required).

2.1 Frontend (React 19 + Vite)

Component	File	Type	Description
App Shell	`src/App.jsx`	Layout	Root component; header, progress bar, routes between form and guide
Trip Context	`src/context/TripContext.jsx`	Context	Global state: form fields, generation, verification, swap modal, budget selections (hotel/flight switching with live total recalc)
Form Steps	`src/components/FormSteps.jsx`	Wizard	9-step trip planning form (origin, destination, dates, budget, vibe, extras, review)
Trip Guide	`src/components/TripGuide.jsx`	Display	Multi-tab itinerary: Map, Itinerary, Stays, Flights, Budget, Suggestions, Tips. Filters "Search Links" fallback entries. Budget tab has selectable options with live price delta.
Verification Bar	`src/components/VerificationBar.jsx`	Widget	Real-time agent completion progress bar
Price Trends	`src/components/PriceTrends.jsx`	Chart	12-month flight & hotel price trend graph; frontend seasonal fallback if backend returns no data
Swap Modal	`src/components/SwapModal.jsx`	Modal	Hotel/restaurant swap with budget impact
Currency Toggle	`src/components/CurrencyToggle.jsx`	Widget	Currency selector + live conversion
PDF Export	`src/components/PdfExport.jsx`	Export	Lazy-loads `@react-pdf/renderer` (∼1.5 MB)
Safety Badge	`src/components/SafetyBadge.jsx`	Badge	Safety level with emoji & color
Rating Badge	`src/components/RatingBadge.jsx`	Badge	Star rating with review count
Seasonal Panel	`src/components/SeasonalPanel.jsx`	Panel	Temperature, rain, tourism level, local tips
Loading View	`src/components/LoadingView.jsx`	Screen	Animated loading with streaming text
Visa Checker	`src/components/VisaChecker.jsx`	Widget	Passport + destination visa requirements
API Client	`src/api.js`	Module	All fetch calls: `askClaudeStream`, `requestVerification`, `searchHotels`
Prompts	`src/prompts.js`	Module	Claude prompt builders: skeleton + day-by-day
Constants	`src/constants.js`	Module	Image URLs, trip types, dietary options, URL builders (Airbnb/Booking.com with specific hotel names)

2.2 Node.js Backend (Express)

Endpoint	File	Rate Limit	Description
`POST /api/chat/stream`	`server.js`	10/min	Server-Sent Events (SSE) proxy to Anthropic Claude API
`POST /api/verify/stream`	`server.js`	5/min	Server-Sent Events (SSE) proxy to Python FastAPI verification
`POST /api/hotels/search`	`server.js`	5/min	On-demand hotel search (swap feature)
`GET /api/health`	`server.js`	—	Health check with API key status
`POST /api/restaurants/alternatives`	`server.js`	10/min	Claude-powered restaurant swap suggestions
`POST /api/clear-cache`	`server.js`	—	Clear Python agent cache
`GET /api/verify/health`	`server.js`	—	Python service health check

2.3 Python Verification Service (FastAPI)

Component	File	Type	Description
FastAPI App	`agents/main.py`	Entry	`/verify`, `/verify/stream`, `/hotels/search`, `/health`
Orchestrator	`agents/orchestrator.py`	Dispatch	`asyncio.gather()` over 11 agents; 45 s timeout (180 s for price trends)
Models	`agents/models.py`	Schema	Pydantic models: `VerifyRequest`, `VerifyResponse`, all result types
Cache	`agents/cache.py`	Storage	SQLite async cache with per-agent TTL
Config	`agents/config.py`	Config	`AGENTS` dict (enabled, provider, cache_hours per agent)
API Config	`agents/api_config.py`	Config	Env-var loader: Amadeus, Booking, Viator, Google Places

2.4 Verification Agents

Agent	File (`agents/agents/`)	Cache	Data Path	Output Model
Weather	`weather_agent.py`	3 h	Open-Meteo (free)	`WeatherResult`
Safety	`safety_agent.py`	24 h	travel-advisory.info	`SafetyResult`
Seasonal	`seasonal_agent.py`	168 h	Open-Meteo Climate	`SeasonalResult`
Currency	`currency_agent.py`	1 h	Frankfurter (ECB)	`CurrencyResult`
Places	`place_agent.py`	168 h	Google Places → Nominatim	`PlaceVerification`
Hotels	`hotel_agent.py`	6 h	Booking API → Google Hotels → Booking	`list[HotelResult]`
Activities	`activity_agent.py`	12 h	Viator API → Viator → GYG	`list[ActivityResult]`
Reviews	`review_agent.py`	48 h	Google Places → Google Search	`list[ReviewResult]`
Flights	`flight_agent.py`	1 h	Amadeus → Google Flights scraper	`list[FlightResult]`
Links	`link_agent.py`	24 h	HTTP HEAD (10 concurrent)	`list[LinkCheck]`
Price Trends	`price_trends_agent.py`	24 h	Amadeus/Booking → scrapers (12 mo)	`PriceTrendsResult`

2.5 API Clients (Optional, Key Required)

Provider	File	Env Vars	Functions
Amadeus	`api_clients/amadeus.py`	`AMADEUS_API_KEY` `AMADEUS_API_SECRET`	`search_flights()`, `_get_token()`, `_resolve_iata()`
Booking.com	`api_clients/booking_api.py`	`BOOKING_API_KEY`	`search_hotels()`
Viator	`api_clients/viator_api.py`	`VIATOR_API_KEY`	`search_activities()`
Google Places	`api_clients/google_places.py`	`GOOGLE_PLACES_API_KEY`	`verify_place()`, `get_place_reviews()`

2.6 Scrapers (Default, No Key Required)

Source	File (`agents/scrapers/`)	Method	Notes
Google Flights	`google_flights.py`	Playwright	Price, airline, duration, stops from card DOM
Google Hotels	`google_hotels.py`	Playwright	`[jsname="mutHjb"]` cards, `h2` names, `$` prices
Booking.com	`booking_com.py`	Playwright	Property cards with name, price, rating
Viator	`viator.py`	httpx+BS4	Tour name, price, rating, duration
GetYourGuide	`getyourguide.py`	httpx+BS4	Activity name, price, rating, duration
Google Reviews	`google_reviews.py`	httpx+BS4	Rating, review count, snippets from search
Base Scraper	`base_scraper.py`	Playwright	Shared browser, anti-detection, `new_page()` context manager

2.7 Free APIs (No Key Required)

Service	File	API Endpoint	Functions
Open-Meteo	`free_apis/open_meteo.py`	open-meteo.com	`get_weather()`, `geocode()`
Travel Advisory	`free_apis/travel_advisories.py`	travel-advisory.info	`get_safety()`, `guess_country_code()`
Frankfurter	`free_apis/frankfurter.py`	frankfurter.dev (ECB)	`get_rates()`, `guess_currencies()`
Nominatim	`free_apis/nominatim.py`	nominatim.openstreetmap.org	`verify_place()`, 1 req/sec limit
Seasonal Data	`free_apis/seasonal_data.py`	climate-api.open-meteo.com	`get_seasonal()`, `get_all_tourism_levels()`

3 System Overview

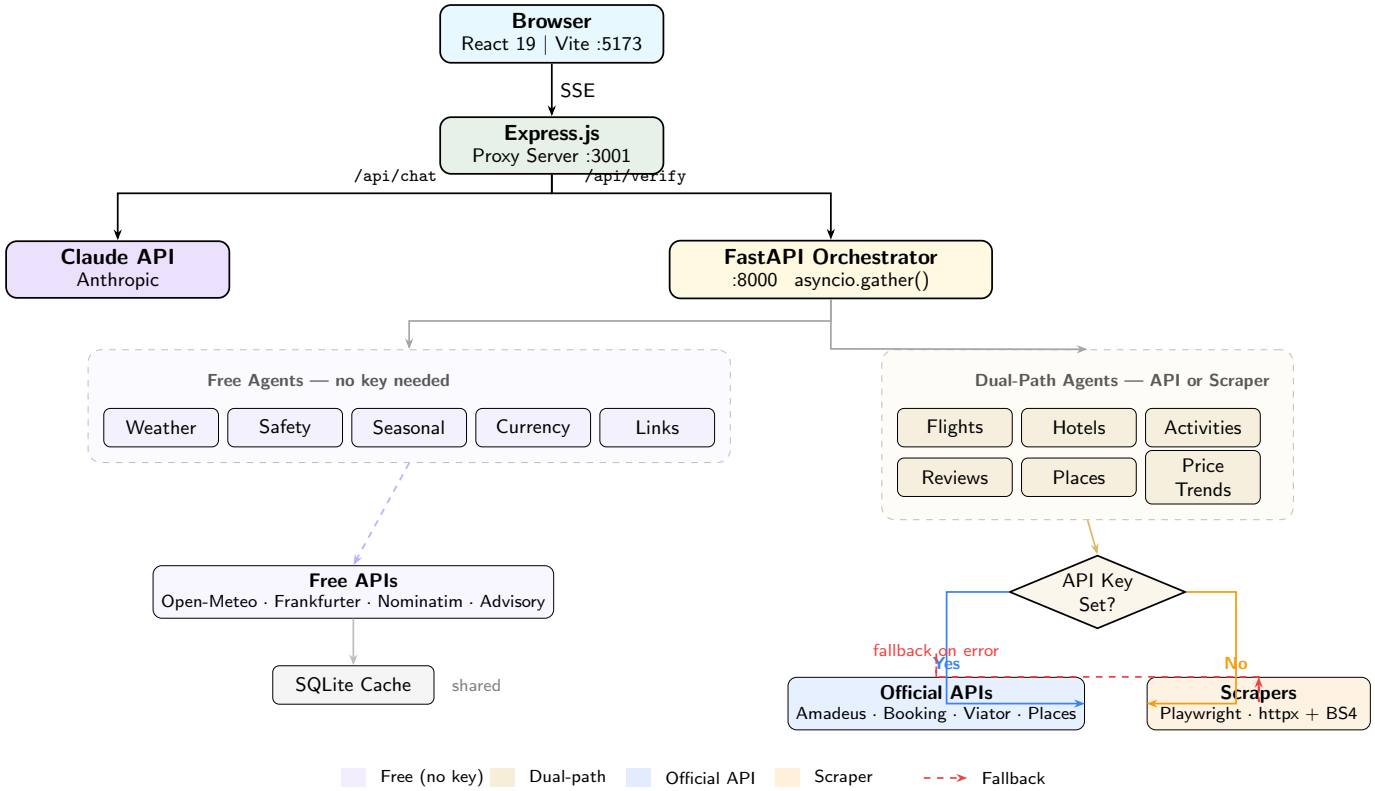

Figure 1: TripMind system architecture. Dual-path agents check for API keys at runtime; the official API is used when a key is present, with scraper fallback on failure.

4 Debug-Mode Execution Flow

This section traces a complete verification request function-by-function. **Blue** = API path, **Orange** = scraper path, **Red** = error/fallback, **Green** = success/cache hit.

4.1 Verification Request Flow

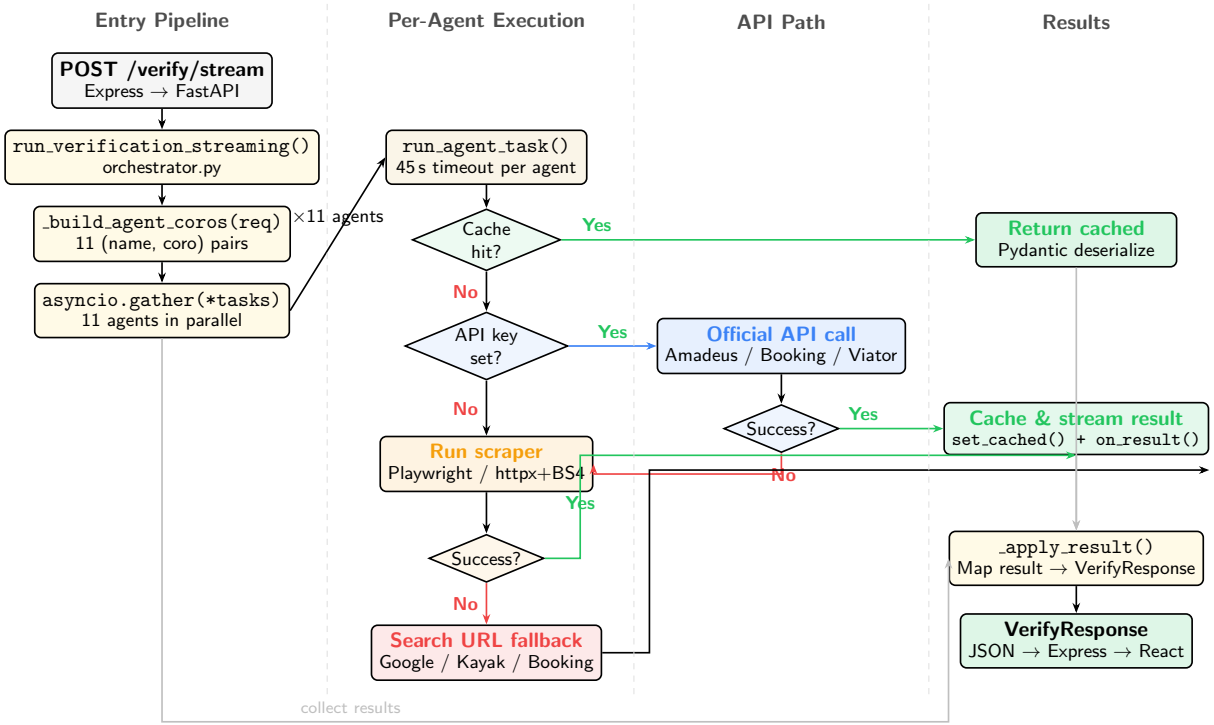

Figure 2: Execution flow for a single dual-path agent. Free-only agents (weather, safety, etc.) skip the API-key check and call their free API directly.

4.2 Function Call Trace (Hotel Agent Example)

Listing 1: Hotel agent execution trace

```
# 1. Entry point (orchestrator.py:250)
async def run_verification_streaming(req, on_result):
    coros = _build_agent_coros(req)   # -> [("hotels",
        run_hotel_agent(...)), ...]

# 2. Agent dispatch (orchestrator.py:59)
async def run_agent_task("hotels", coro):
    result = await asyncio.wait_for(coro, timeout=45)

# 3. Hotel agent (agents/hotel_agent.py:11)
async def run_hotel_agent(city, check_in, check_out, ...):
    cached = await get_cached(cache_key)       # cache.py:get_cached
    if cached: return [HotelResult(**h) for h in cached]

    # Priority 1: Booking.com API
    config = get_booking_config()              # api_config.py
    if config["key"]:
        results = await search_hotels(...)     #
            api_clients/booking_api.py

    # Priority 2: Google Hotels scraper
    if not results:
        results = await scrape_google_hotels(...)  #
            scrapers/google_hotels.py

    # Priority 3: Booking.com scraper
    if not results:
        results = await scrape_booking(...)    # scrapers/booking_com.py

    # Priority 4: Search URL fallback
    if not results:
        results = [HotelResult(name="...", source="Search Links", ...)]

    await set_cached(cache_key, ...)           # cache.py:set_cached
    return results
```

5 API vs Scraper Decision Reference

5.1 Dual-Path Agents

These agents use an official API when the corresponding environment variable is set, falling back to web scraping otherwise.

Agent	Default (Scraper)	API Provider	.env Variables	Fallback Chain
Flights	Google Flights (Playwright)	Amadeus	`AMADEUS_API_KEY`, `AMADEUS_API_SECRET`	Amadeus → Google Flights scraper → search URLs
Hotels	Google Hotels (Playwright)	Booking.com	`BOOKING_API_KEY`	Booking API → Google Hotels → Booking scraper → search URLs
Activities	Viator + GYG (httpx)	Viator	`VIATOR_API_KEY`	Viator API → Viator scraper → GYG → search URLs
Reviews	Google Search (httpx)	Google Places	`GOOGLE_PLACES_API_KEY`	Places API → Google Search scraper
Places	Nominatim (free)	Google Places	`GOOGLE_PLACES_API_KEY`	Places API → Nominatim
Price Trends	Google Flights + Hotels	Amadeus + Booking	All flight + hotel keys	Per-month: API → scraper for flights and hotels

5.2 Free-Only Agents

These agents always use free, keyless APIs. No configuration needed.

Agent	API	Rate Limit	Notes
Weather	Open-Meteo Forecast	10 k/day	7–14 day forecast with WMO codes
Safety	travel-advisory.info	Unlimited	650+ countries, score 0–5
Seasonal	Open-Meteo Climate	10 k/day	30-year climate averages
Currency	Frankfurter (ECB)	Unlimited	ECB daily rates, 30+ currencies
Links	HTTP HEAD requests	N/A	10 concurrent, 8 s timeout each

5.3 Environment Variable Quick Reference

Variable	Required	Description
`ANTHROPIC_API_KEY`	Yes	Claude API key for trip generation
`ANTHROPIC_MODEL`	No	Model override (default: `claude-sonnet-4-20250514`)
`PORT`	No	Express server port (default: 3001)
`PYTHON_SERVICE_URL`	No	FastAPI URL (default: `http://127.0.0.1:8000`)
`AMADEUS_API_KEY`	No	Amadeus client ID (OAuth2)
`AMADEUS_API_SECRET`	No	Amadeus client secret (OAuth2)
`AMADEUS_API_BASE_URL`	No	Override base URL (default: `api.amadeus.com`)
`BOOKING_API_KEY`	No	Booking.com Partner API key
`BOOKING_AFFILIATE_ID`	No	Booking.com affiliate ID (optional)
`VIATOR_API_KEY`	No	Viator Partner API key
`GOOGLE_PLACES_API_KEY`	No	Google Places API key (reviews + places)

5.4 Data Contract Summary

All API clients and scrapers return identical Pydantic models. Normalization happens inside each data source, not downstream.

Model	Key Fields	Sources
`FlightResult`	airline, price, duration, stops, booking_url	`api_clients/amadeus.py`, `scrapers/google_flights.py`
`HotelResult`	name, price_per_night, rating, booking_url	`api_clients/booking_api.py`, `scrapers/google_hotels.py`, `scrapers/booking_com.py`
`ActivityResult`	name, price, rating, duration, booking_url	`api_clients/viator_api.py`, `scrapers/viator.py`, `scrapers/getyourguide.py`
`ReviewResult`	place_name, rating, review_count, snippets	`api_clients/google_places.py`, `scrapers/google_reviews.py`
`PlaceVerification`	query, found, lat/lon, confidence	`api_clients/google_places.py`, `free_apis/nominatim.py`
`WeatherResult`	days (temp, precip, code, emoji)	`free_apis/open_meteo.py`
`SafetyResult`	score, level, emoji, advice, alerts	`free_apis/travel_advisories.py`
`SeasonalResult`	season, tourism_level, insights	`free_apis/seasonal_data.py`
`CurrencyResult`	rates dict (rate, inverse, date)	`free_apis/frankfurter.py`
`PriceTrendsResult`	monthly_prices (12 mo flight+hotel)	Reuses flight + hotel agents
`LinkCheck`	url, status, reachable	`agents/link_agent.py` (HTTP HEAD)

6 Frontend Data Quality Controls

When scrapers fail (blocked by anti-bot, JS not rendered, etc.), agents return "Search Links" fallback entries with generic names like "Hotels in {city}" and null prices. The frontend applies several filters to prevent these from reaching the user.

6.1 Search Links Filtering

Location	Filter	Behavior when all filtered out
Stays tab (verified hotels)	`source !== "Search Links" && price_per_night`	Falls through to AI Suggested Options with name-specific Airbnb/Booking.com links
Budget tab (hotel options)	Same filter; require ≥2 remaining	Section hidden entirely; budget uses AI estimates
Budget tab (flight options)	`price && source !== "Search Links"`; require ≥2	Section hidden
Flights tab (verified)	`source !== "Search Links"`	Shows only AI Estimated Flights
Activities (verified match)	`source !== "Search Links"`	Shows AI Estimate badge instead of verified price

6.2 Price Trends Frontend Fallback

The `PriceTrends.jsx` component guarantees charts are never empty:

1. Try backend data: `verification.price_trends[destination].monthly_prices`
2. If backend returned null/zero prices for all months, generate client-side estimates using seasonal shape curves (same `MONTH_SHAPE` multipliers as the Python backend)
3. Fallback baselines: $450/flight, $120/hotel per night, scaled by travel month ratio
4. When using fallback, displays "(seasonal estimates)" label and adjusted disclaimer

6.3 Budget Selection System

The Budget tab provides interactive hotel and flight option cards when verified data has ≥2 real results:

- **selectHotelOption(location, index)**: Switches selected hotel for a location, recalculates accommodation row in budget breakdown, updates total and per-person estimates
- **selectFlightOption(index)**: Same for flights, multiplied by passenger count
- **resetBudgetSelections()**: Restores original AI-estimated budget values
- Delta badges show price difference (green = savings, red = more expensive) relative to currently selected option

6.4 Booking Link Specificity

URL builders in `constants.js` accept an optional `name` parameter to generate property-specific search links instead of generic city searches:

- `airbnbUrl(city, ci, co, adults, children, maxB, name)` — searches "{name}, {city}"
- `bookingUrl(city, ci, co, adults, name)` — same pattern
- Activity links use `bookingKeyword + location` for Viator/GYG/TripAdvisor searches

- Verified activities show direct `booking_url` when the scraper/API found an exact match

A Detailed Agent Reference

This appendix documents every verification agent: its function signature, data sources, fallback chain, caching strategy, and notable implementation details.

A.1 Weather Agent

Weather — Free / No Key

Function	`run_weather_agent(location, check_in, check_out)`
Data Source	Open-Meteo Forecast API (free, 10 k requests/day)
Cache Key	`weather:{location}:{check_in}:{check_out}`
Cache TTL	3 hours (config-driven)
Output Model	`WeatherResult` — days array with temp_max, temp_min, precip_mm, weather_code, emoji
Inputs Used	`location` (city name), `check_in`/`check_out` (ISO dates)

Behavior: Geocodes the location via Open-Meteo, then fetches a 7–14 day forecast. Returns daily temperature, precipitation probability, and WMO weather codes mapped to emojis. Only caches when the days array is non-empty and error-free.

A.2 Safety Agent

Safety — Free / No Key

Function	`run_safety_agent(destination, passport_country="")`
Data Source	travel-advisory.info API (free, unlimited)
Cache Key	`safety:{dest_lower}:{passport_lower}`
Cache TTL	24 hours
Output Model	`SafetyResult` — score (0–5), level, emoji, advice text, active alerts
Inputs Used	`destination`, `passport_country` (from form)

Behavior: Normalizes inputs to lowercase before cache lookup and API call. The passport_country is passed to the advisory API for country-specific context (e.g., visa warnings). Scores: 0 = safest, 5 = avoid travel.

A.3 Seasonal Agent

Seasonal — Free / No Key

Function	`run_seasonal_agent(destination, check_in, check_out, country_code="")`
Data Source	Open-Meteo Climate API (free, 10 k/day; 30-year averages)
Cache Key	`seasonal:{dest_lower}:{check_in[:7]}`
Cache TTL	168 hours (7 days)
Output Model	`SeasonalResult` — season classification, tourism_level, climate insights, local events
Inputs Used	`destination`, `check_in`, `check_out`, `country_code`

Behavior: Cache key uses only `YYYY-MM` from check_in for broad matching across dates in the same month. If country_code is not provided, it is guessed from the destination string. Returns 30-year climate averages rather than forecasts.

A.4 Currency Agent

Currency — Free / No Key

Function	`run_currency_agent(base_currency, destinations)`
Data Source	Frankfurter API (free, ECB daily rates, 30+ currencies)
Cache Key	`currency:{base}:{sorted_dests_joined}`
Cache TTL	1 hour
Output Model	`CurrencyResult` — rates dict with rate, inverse_rate, date per currency
Inputs Used	`base_currency` (e.g., USD), `destinations` (city/country list)

Behavior: Sorts the destination list alphabetically before cache key generation to ensure consistent caching regardless of input order. Guesses local currencies from city/country names. Returns both forward and inverse exchange rates.

A.5 Hotel Agent

Hotels — Dual-Path

Function	`run_hotel_agent(city, check_in, check_out, adults=2, children=0, max_price=0, currency="USD")`
Fallback Chain	Booking API → Google Hotels scraper → Booking.com scraper → Search URLs
Cache Key	`hotels:{city}:{ci}:{co}:{adults}:{children}:{max_price}`
Cache TTL	6 hours
Output Model	`list[HotelResult]` — name, source, price_per_night, rating, stars, booking_url, image_url
Inputs Used	`city`, `check_in/out`, `adults`, `children_ages` (length), `max_nightly`, `currency`

Fallback details:

1. **Booking.com API** — requires `BOOKING_API_KEY`; returns structured results with ratings and images
2. **Google Hotels scraper** — Playwright; parses `[jsname="mutHjb"]` card selectors (fragile, Google changes frequently)
3. **Booking.com scraper** — Playwright; parses property card DOM elements
4. **Search URLs** — generates Booking.com + Airbnb links with encoded city/dates; `source="Search Links"`, `price_per_night=null`; always succeeds but filtered by frontend

Only non-search-URL results are cached. The `children_ages` array length determines group size for scraper queries.

A.6 Activity Agent

Activities — Dual-Path

Function	`run_activity_agent(activity_name, city, currency="USD")`
Fallback Chain	Viator API → Viator scraper (3 results) → GetYourGuide scraper (2 results) → Search URLs
Cache Key	`activity:{activity_name}:{city}`
Cache TTL	12 hours
Output Model	`list[ActivityResult]` — name, matched_activity, price, rating, duration, booking_url, source
Inputs Used	`activity_name` (from Claude plan), `city`, `currency`

Behavior: GetYourGuide results are merged with Viator results, deduplicating by lowercase name to avoid showing the same tour twice. Only non-search-URL results are cached. The `matched_activity` field links the result back to the original activity name from Claude's plan.

A.7 Review Agent

Reviews — Dual-Path

Function	`run_review_agent(places: list[dict])`
Fallback Chain	Google Places API → Google Search scraper
Cache Key	`review:{name_lower}:{city_lower}` (per-place)
Cache TTL	48 hours
Output Model	`list[ReviewResult]` — place_name, rating (float), review_count, url, snippets, error
Inputs Used	`places` list of {name, city} dicts; capped at 10

Behavior: Each place is cached independently. Google Search scraper is used as fallback if the Places API returns a null rating. Only caches results where `rating` is not None and no error occurred. Processes up to 10 places per request.

A.8 Flight Agent

Flights — Dual-Path

Function	`run_flight_agent(origin, destination, departure_date, return_date="", passengers=1, currency="USD")`
Fallback Chain	Amadeus API → Google Flights scraper → Search URLs (3 sites)
Cache Key	`flights:{origin}:{dest}:{dep}:{ret}:{pax}`
Cache TTL	1 hour
Output Model	`list[FlightResult]` — airline, price, departure, arrival, duration, stops, booking_url, source
Inputs Used	`origin`, `destination`, `departure_date`, `return_date`, `passengers` (adults + children_ages length)

Behavior: Amadeus API uses OAuth2 with automatic token refresh. IATA codes resolved via Amadeus location API. If all real sources fail, generates search URLs for Google Flights, Kayak, and Skyscanner. Only non-search-URL results cached. The `passengers` count includes both adults and children.

A.9 Place Agent

Places — Dual-Path

Function	`run_place_agent(name, city="")`
Fallback Chain	Google Places API → Nominatim/OpenStreetMap
Cache Key	`place:{name}:{city}`
Cache TTL	168 hours (7 days)
Output Model	`PlaceVerification` — query, found (bool), lat, lng, address, confidence, error
Inputs Used	`name` (place/attraction name), `city`

Behavior: Verifies whether a place mentioned in Claude's plan actually exists. Nominatim is always tried as fallback even if Google Places API returns "not found." Nominatim enforces a 1 request/second rate limit. Batch-processes up to 20 places per verification request.

A.10 Link Agent

Links — Free / No Key

Function	`run_link_agent(urls: list[str])`
Check Method	HTTP HEAD requests via httpx (concurrent)
Cache Key	`link:{url}` (per-URL)
Cache TTL	24 hours
Output Model	`list[LinkCheck]` — url, status (int), reachable (bool), redirect_url, error
Inputs Used	All URLs extracted from Claude's generated plan (capped at 50)

Behavior: Runs up to 10 concurrent HEAD requests using an asyncio semaphore. Each request has an 8-second timeout. A link is "reachable" if it returns a 2xx–3xx status code. Invalid URLs are rejected early without making a request. Follows redirects automatically.

A.11 Price Trends Agent

Price Trends — Dual-Path (Composite)

Function	`run_price_trends_agent(origin, destination, check_in, check_out, adults=2, children=0, currency="USD", country_code="")`
Strategy	Baseline scrape + seasonal multiplier estimation (2 scrapes, not 24)
Cache Key	`ptrends:{origin}:{dest}:{adults}:{children}`
Cache TTL	24 hours
Output Model	`PriceTrendsResult` — monthly_prices (12 × MonthlyPrice: month_name, avg_flight, avg_hotel, tourism_level)
Inputs Used	`origin`, `destination`, `check_in/out`, `adults`, `children_ages` (length), `currency`, `country_code`

Three-step algorithm:

1. **Baseline fetch** — scrape flight and hotel prices for the travel month only (Amadeus API → Google Flights; Booking API → Google Hotels; 35 s timeout each)
2. **Seasonal estimation** — multiply baseline by per-month shape curves:
   - Tourism multipliers: peak = 1.35, shoulder = 1.05, off-peak = 0.75
   - Hemisphere-aware: southern countries (AU, NZ, AR, CL, BR, ZA, etc.) use inverted seasonal curves
   - Fallback baselines if scraping fails: $450/flight, $120/hotel per night
3. **Cache validation** — skips stale entries where all prices are null

This approach avoids 24 separate scrapes (12 months × flights + hotels) while still producing reasonable month-by-month estimates.

B Orchestrator Internals

The orchestrator (`agents/orchestrator.py`) is the central dispatcher that runs all agents concurrently and streams results back to the frontend via Server-Sent Events (SSE).

B.1 Dispatch Pipeline

1. `run_verification_streaming(req, on_result)` — Main entry point. Builds agent coroutines, dispatches them in parallel, applies results, and calls `on_result` after each agent completes.
2. `_build_agent_coros(req)` — Constructs a list of `(name, coroutine)` pairs:
   - **Per-destination agents**: Weather, Safety, Seasonal, Hotels — one coroutine per destination (e.g., `"weather:Paris"`, `"hotels:Tokyo"`)
   - **Single-destination agents**: Flights, Price Trends — primary destination only
   - **Batch agents**: Places (up to 20), Activities (up to 15), Reviews (up to 10), Links (up to 50 URLs)
   - **Global agents**: Currency (single call with all destinations)

   Destinations extracted from `req.destinations` or `trip_plan.allDestinations`. Primary destination from `trip_plan.primaryDestination`.
3. `run_agent_task(name, coro)` — Wraps each coroutine with timeout enforcement:
   - Price trends: 60 s timeout (needs baseline scrape + estimation)
   - All other agents: 45 s timeout
   - On timeout: returns error result, does not crash other agents
4. `asyncio.gather(*tasks, return_exceptions=True)` — All agent tasks run in parallel. Exceptions are captured, not raised, so one failing agent does not block others.
5. `_apply_result(response, name, result)` — Maps agent name to the correct field on `VerifyResponse`:

Agent Name Pattern	Response Field	Type
`weather:{dest}`	`response.weather[dest]`	WeatherResult
`safety:{dest}`	`response.safety[dest]`	SafetyResult
`seasonal:{dest}`	`response.seasonal[dest]`	SeasonalResult
`currency`	`response.currency`	CurrencyResult
`hotels:{dest}`	`response.hotels[dest]`	list[HotelResult]
`flights`	`response.flights`	list[FlightResult]
`activities`	`response.activities`	list[ActivityResult]
`reviews`	`response.reviews`	list[ReviewResult]
`places`	`response.places`	list[PlaceVerification]
`links`	`response.links`	list[LinkCheck]
`price_trends:{dest}`	`response.price_trends[dest]`	PriceTrendsResult

B.2 Server-Sent Events (SSE) Streaming

Server-Sent Events (SSE) is a standard HTTP mechanism that allows a server to push real-time updates to a client over a single, long-lived connection. Unlike WebSockets, SSE is unidirectional (server-to-client only), uses plain HTTP, and automatically reconnects on failure. Trip-

Mind uses SSE in two places: Claude trip generation (`/api/chat/stream`) and agent verification (`/api/verify/stream`).

The `on_result(name, status, response)` callback is invoked after each agent completes. This enables real-time progress updates in the frontend:

1. The FastAPI endpoint `/verify/stream` creates a `StreamingResponse` with `content-type: text/event-stream`
2. As each agent finishes, `on_result` emits an SSE event containing the agent name, completion status, and the partial `VerifyResponse` accumulated so far
3. The React frontend receives these events via the browser's built-in `EventSource` API; the `VerificationBar.jsx` component updates its progress bar in real time as each agent reports in
4. When all agents have completed, a final SSE event delivers the complete `VerifyResponse` with all results, and the connection closes

B.3 Verification Score

After all agents complete, the orchestrator computes a verification score:

$$\text{verification_score} = \frac{\text{successful agents}}{\text{total agents}} \times 100$$

An agent is "successful" if it returns a result without an error field. Agents that time out or throw exceptions are counted as failures but do not prevent the response from being sent.

B.4 Parameter Pass-Through

Key parameters from the trip form are routed to specific agents:

Form Field	Agents	Usage
`children_ages`	Hotels, Flights	Array length → group size / passenger count
`passport_country`	Safety	Travel advisory context
`currency`	Hotels, Flights, Activities, Currency, Price Trends	Price denomination
`max_nightly`	Hotels	Max price filter
`check_in` / `check_out`	Weather, Seasonal, Hotels, Flights, Price Trends	Date range for queries
`dietary`, `accessibility`, `petFriendly`, `tripTypes`, `activityLevel`	Claude prompts only	No agent can filter by these